Universidad de Las Palmas de Gran Canaria
Grado en Ciencia e Ingeniería de Datos

Big Data - Academic Year 2026/2027

Distributed Matrix Multiplication with MPI

Individual Assignment 4

Sara Carrizosa Tomé

GitHub Repository

November 2026

Abstract

This experiment investigates the performance of distributed matrix multiplication using MPI in C. The implementation distributes matrix rows among processes and uses collective communication to exchange data and collect the result. Five matrix sizes, from 100 to 1000, were tested with 1, 2, 4 and 8 MPI processes.

Execution time, memory usage and communication costs were measured. The results show that increasing the number of processes generally improves performance for larger matrices, although the speedup remains below the ideal value. For a matrix of size 1000, execution time decreased from 341.60 ms with one process to 91.52 ms with eight processes, giving a speedup of 3.73. Smaller matrices benefited less from additional processes.

The experiments were conducted on a single computer, so the results describe local multi-process execution rather than communication between physical machines.

1 Introduction

Matrix multiplication is a computationally expensive operation used in many scientific and data processing applications. For two dense square matrices of size $n \times n$, the standard algorithm requires $O(n^3)$ arithmetic operations.

Previous assignments explored basic implementations, sequential optimisations and shared-memory parallelism with OpenMP. This assignment extends that investigation by implementing matrix multiplication with the Message Passing Interface (MPI).

Unlike OpenMP, where threads operate within a shared address space, MPI processes have separate memory spaces and exchange data through explicit communication operations.

The main research question is:

> How does the number of MPI processes affect matrix multiplication performance, memory usage and communication overhead?

The experiment also compares MPI with the sequential and OpenMP implementations from Assignment 3.

It was expected that additional processes would reduce computation time, particularly for larger matrices. However, communication and process overhead could limit the improvement, especially when the amount of work assigned to each process is small.

2 Background

2.1 Distributed Matrix Multiplication

The implementation uses a row-based decomposition. Matrix A is divided into groups of rows, while matrix B is made available to every process.

Each process computes the rows of the result matrix corresponding to its assigned portion of A.

For matrices A and B, each element of the result is calculated as

$$C_{ij} = \sum_{k=0}^{n-1} A_{ik} B_{kj}. \tag{1}$$

The multiplication follows the same `i-k-j` loop order used in the previous assignments. This keeps the arithmetic algorithm comparable across implementations.

MPI provides collective communication operations that allow the input data and results to be distributed and collected [1].

2.2 MPI Communication

Four main operations are used in the implementation.

- `MPI_Bcast`: distributes matrix B from the root process to the others.
- `MPI_Scatterv`: distributes different groups of rows from matrix A.
- `MPI_Gatherv`: collects the partial results into matrix C.
- `MPI_Barrier`: synchronises processes at selected measurement boundaries.

The use of `Scatterv` and `Gatherv` allows uneven row distributions when the matrix size is not divisible by the number of processes.

2.3 Data Distribution Costs

The communication volume depends on the matrix size and the number of MPI processes.

Each matrix contains n^2 integer elements. Since the implementation uses 32-bit integers, each matrix requires approximately $4n^2$ bytes.

During Broadcast, matrix B is distributed to the other $p-1$ processes. The total logical amount of data delivered to the receiving processes is therefore

$$V_{\text{broadcast}} = 4n^2(p-1). \tag{2}$$

For Scatter and Gather, the rows assigned to the non-root processes must be transferred. The corresponding logical communication volume for each operation is

$$V_{\text{scatter}} = V_{\text{gather}} = 4n^2 \left(1 - \frac{r_0}{n}\right), \tag{3}$$

where r_0 is the number of rows assigned to the root process.

For $n = 1000$, a complete matrix contains 4 MB of integer data. These estimates describe the logical data volume and do not include MPI protocol overhead or additional internal copies.

Since all processes ran on one computer, the measurements reflect local communication rather than network transfers between machines.

2.4 Scaling Metrics

Strong scaling is investigated by keeping the matrix size fixed while increasing the number of processes.

The speedup is defined as

$$S(p) = \frac{T(1)}{T(p)}, \tag{4}$$

where $T(1)$ is the median MPI execution time with one process and $T(p)$ is the median with p processes.

Parallel efficiency is calculated as

$$E(p) = \frac{S(p)}{p} \times 100. \tag{5}$$

These metrics indicate how effectively additional processes reduce execution time.

3 Methodology

3.1 Experimental Environment

All MPI experiments were performed on a single Windows computer. This configuration was permitted by the instructor.

The experimental environment was:

- CPU: Intel Core Ultra 7 155H
- Physical cores: 16
- Logical processors: 22
- RAM: approximately 32 GiB
- Operating system: Windows 11 Home
- Programming language: C
- Compiler: GCC 16.2.0
- Compiler flags: `-O2 -std=c11`
- MPI implementation: Microsoft MPI 10.1

The implementation was compiled using MSYS2 UCRT64, with the Windows PSAPI library linked for memory measurements.

No configuration exceeded the 22 logical processors available on the computer. Since all MPI processes ran locally, the experiment does not evaluate inter-machine network communication.

3.2 Implementation

The distributed program was implemented in `distributed.c`.

Process 0 reads matrices A and B from CSV files. Matrix B is then broadcast to all processes, while the rows of A are distributed using `MPI_Scatterv`.

Each process executes the multiplication locally and stores its result in a separate buffer. The partial results are collected with `MPI_Gatherv`.

When the rows cannot be divided equally, the remaining rows are assigned to the first processes.

The implementation uses integer matrices and the same multiplication loop order as the sequential and OpenMP versions.

3.3 Input Data and Correctness

The input matrices were reused from the previous assignments.

Five square matrix sizes were selected:

$$n \in \{100, 300, 500, 750, 1000\}. \tag{6}$$

The input matrices were generated using Python's `random.Random` generator, with fixed seeds 42 for matrix A and 43 for matrix B. Their elements are integers uniformly sampled from the range $[0, 9]$.

The smaller matrices were generated in Assignment 1 using `scripts/generate_inputs.py`, while the larger matrices were generated using the corresponding script from Assignment 2.

The C implementation uses 32-bit signed integers. For the largest tested size, $n = 1000$, the maximum possible value of an output element is $1000 \times 9^2 = 81000$, which is well below the 32-bit signed integer limit. Therefore, integer overflow is not expected for these inputs.

The largest tested matrix size was $n = 1000$, which was also the maximum supported by the available input datasets and the implementation. No larger configurations were attempted.

The selected sizes allowed the experiment to compare small workloads, where process overhead can have a greater impact, with larger workloads that provide more computation per process.

For each size, the same input matrices were used across the different MPI process configurations.

The program compares the resulting matrix against the expected CSV output from the earlier assignments.

Validation is performed after every multiplication, including warm-up runs. All configurations included in the recorded benchmark completed successfully and passed validation.

The experiment was limited to the available matrix sizes, with $n = 1000$ as the largest tested and attempted size. No explicit numerical time or memory budget was established before benchmarking. All selected configurations completed successfully without reaching a resource limit. This is a limitation of the experimental planning.

3.4 Benchmark Protocol

Four process configurations were evaluated:

$$p \in \{1, 2, 4, 8\}. \tag{7}$$

For each configuration, one warm-up run was performed, followed by five measured repetitions.

The median was used to represent execution time, while the interquartile range (IQR) was used to describe variability.

The IQR was calculated as the difference between the fourth and second ordered observations for the five measured samples.

All individual measurements were saved in CSV files for later analysis.

3.5 Execution Time Measurements

The distributed computation was measured using `MPI_Wtime()`.

The measured region includes the broadcast, scatter, local multiplication, gather and synchronisation operations.

Input loading, initial memory allocation and correctness validation are outside this region. The initialisation of the local result buffer also takes place before timing begins.

Each process measures its own execution time. The maximum elapsed time across processes is reported for each measured phase.

Individual phase measurements may include time spent waiting for other processes. Furthermore, the maxima can belong to different processes, so they cannot be added together to reconstruct the total execution time.

3.6 Memory Measurements

Memory usage was measured using the Windows `GetProcessMemoryInfo()` function [2].

For each MPI process, the peak working set was obtained and expressed in MiB.

Two values were recorded:

- The sum of the individual process peak working sets.
- The maximum peak working set observed for a single process.

The sum is not necessarily the maximum simultaneous memory consumption because different processes may reach their peaks at different moments.

The measurements describe the MPI application processes and do not account for all memory used by the MPI runtime or operating system.

3.7 End-to-End Measurements

A separate Python script measured the total elapsed time from launching `mpiexec` until the program finished.

This includes MPI initialisation, CSV loading, memory allocation, one internal warm-up run, five measured multiplications, validation and program termination.

For each configuration, five complete program executions were measured after an additional unrecorded warm-up execution.

These results were stored separately because one end-to-end measurement represents a complete benchmark session rather than a single multiplication.

4 Results

4.1 Execution Time

Figure 1 shows the median MPI execution time for each matrix size and process configuration.

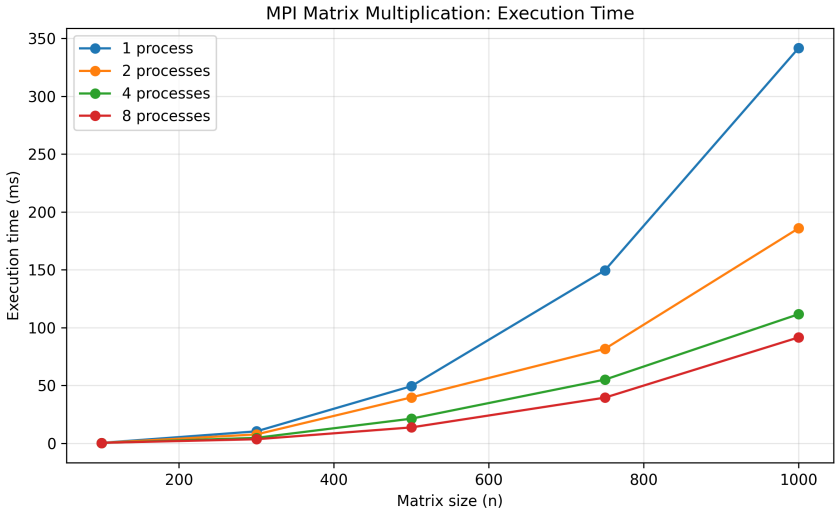


Figure 1: Median MPI execution time for different matrix sizes and process counts.

Execution time increases with matrix size in every configuration. However, additional processes generally reduce the time required for larger matrices.

For $n = 1000$, the measured results were:

Table 1: MPI execution time for $n = 1000$.

Processes	Median (ms)	IQR (ms)	Speedup	Efficiency (%)
1	341.60	12.24	1.00	100.00
2	185.84	4.58	1.84	91.91
4	111.57	5.45	3.06	76.55
8	91.52	10.88	3.73	46.66

The largest reduction occurs when increasing the number of processes from one to four. Moving from four to eight still improves performance, but the additional gain is smaller.

For $n = 100$, the execution times were much lower. The median decreased from 0.3524 ms with one process to 0.2271 ms with four processes, but increased to 0.3843 ms with eight processes.

This indicates that additional MPI processes do not always improve performance for small computational workloads.

The complete timing results, including the median and IQR for every tested configuration, are provided in Table 3.

4.2 Speedup and Efficiency

Figure 2 presents the speedup and efficiency obtained with MPI.

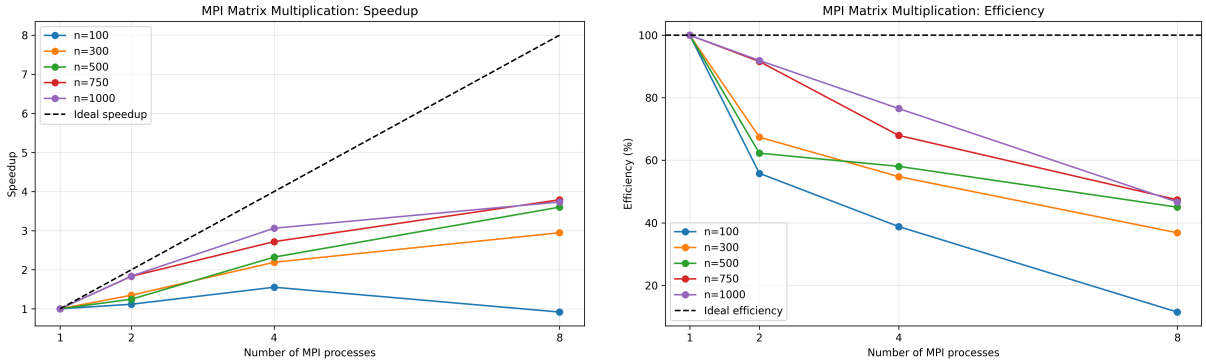


Figure 2: MPI speedup and efficiency relative to the one-process implementation.

Larger matrices generally achieve better speedup because they provide more computational work for each process.

For $n = 1000$, eight processes achieve a speedup of 3.73, compared with the ideal value of 8.

Efficiency decreases as the number of processes increases, reaching approximately 46.66% with eight processes.

For $n = 100$, the speedup with eight processes falls below 1, meaning that this configuration is slower than the one-process implementation.

The results show that scalability depends on the balance between useful computation and process-related overhead.

4.3 Computation and Communication

Figure 3 presents the median computation and communication phase durations for $n = 1000$.

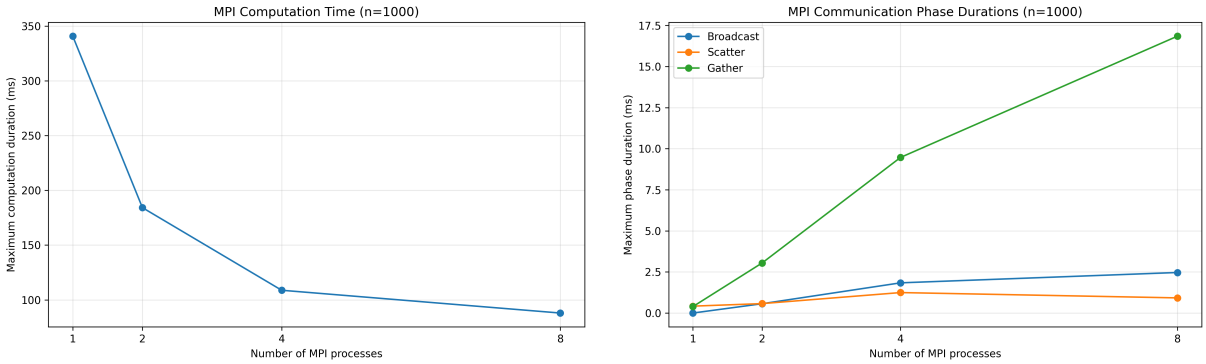


Figure 3: Computation time and MPI collective operation durations for $n = 1000$.

The maximum computation duration decreases from 340.77 ms with one process to 87.98 ms with eight processes.

The communication operations show a different behaviour.

For eight processes, the median maximum durations were approximately 2.47 ms for Broadcast, 0.92 ms for Scatter and 16.85 ms for Gather.

Gather presents the highest reported phase duration and increases substantially as more processes are used.

However, these measurements are not pure network transfer times. Collective operations can involve waiting for processes that have not reached the same stage of execution.

For this reason, the measurements are interpreted as observed phase durations rather than isolated communication costs.

4.4 Memory Usage

Figure 4 shows how memory usage changes with the number of MPI processes.

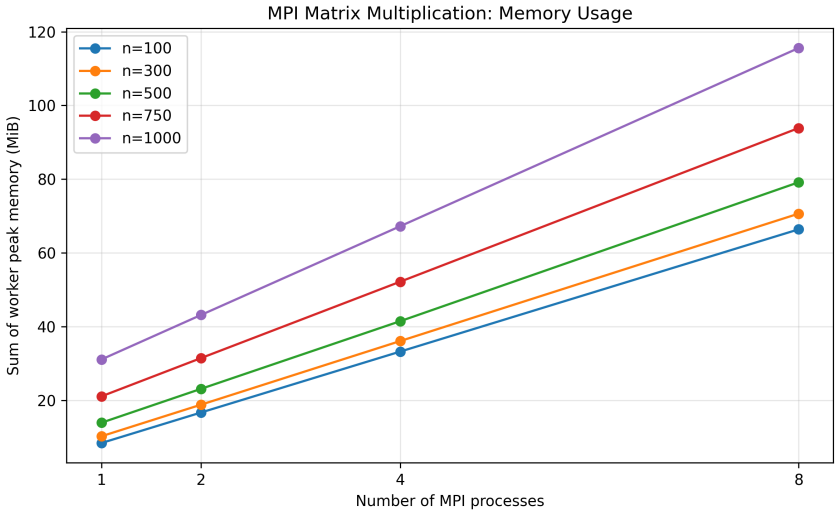


Figure 4: Sum of individual MPI process peak working sets for different matrix sizes.

Memory usage increases as more processes are created.

For $n = 1000$, the sum of individual peak working sets increases from 31.10 MiB with one process to 115.57 MiB with eight processes.

At the same time, the maximum peak working set of a single process decreases from 31.10 MiB to 24.59 MiB.

This behaviour is consistent with the distribution of matrix rows among processes, while each process still requires its own memory space and a copy of matrix B.

Although the amount of work assigned to each process decreases, the overall memory cost increases due to replicated data and process-related overhead.

4.5 Comparison with Sequential and OpenMP

The MPI results were compared with the sequential and OpenMP implementations from Assignment 3.

Figure 5 presents the measured execution times.

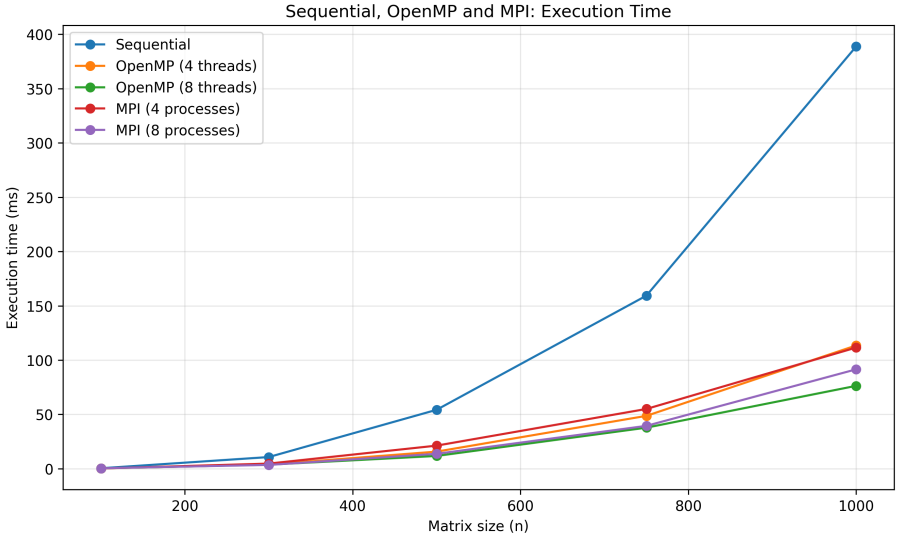


Figure 5: Execution time comparison between Sequential, OpenMP and MPI implementations.

For $n = 1000$, the results were:

Table 2: Execution time comparison for $n = 1000$.

Implementation	Median time (ms)
Sequential	388.85
OpenMP, 4 threads	113.51
OpenMP, 8 threads	76.32
MPI, 4 processes	111.57
MPI, 8 processes	91.52

The one-process MPI implementation also recorded a lower execution time than the sequential version from Assignment 3 (341.60 ms compared with 388.85 ms). Since both implementations use the same multiplication algorithm, this difference may be related to implementation or measurement conditions. The available results do not allow its exact cause to be identified.

Both parallel approaches achieve lower execution times than the sequential baseline for this size.

OpenMP with eight threads obtains the lowest measured time, while MPI with eight processes is slower under the tested configuration.

With four workers, the recorded times are similar, with MPI slightly faster for $n = 1000$.

However, the comparison has limitations. The MPI measurements include distribution, collection and synchronisation, whereas the sequential and OpenMP implementations use different timing boundaries.

Therefore, these results describe the performance of the tested implementations, but do not provide an isolated measurement of MPI versus OpenMP framework overhead.

4.6 End-to-End Performance

Figure 6 presents the time required to execute the complete MPI program.

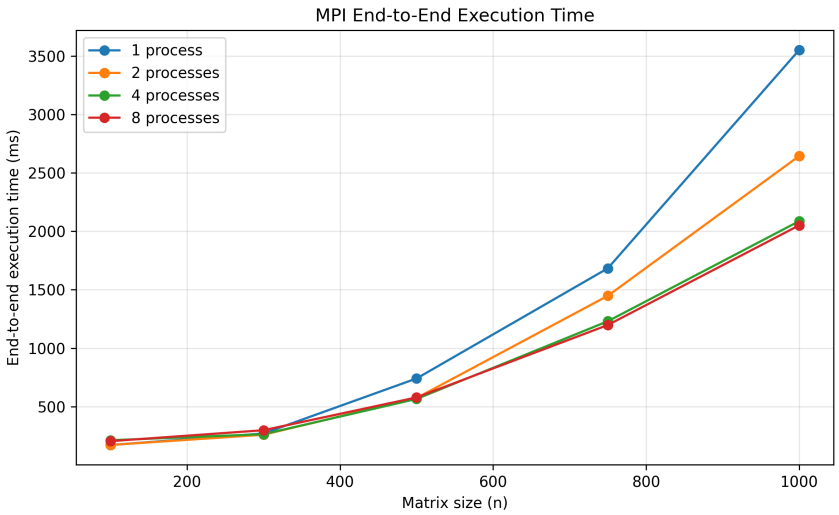


Figure 6: Median end-to-end execution time for complete MPI benchmark sessions.

For $n = 1000$, the median end-to-end times were 3550.65 ms with one process, 2643.69 ms with two, 2086.51 ms with four and 2050.67 ms with eight processes.

The improvement from four to eight processes is relatively small compared with the reduction observed when moving from one to four processes.

For smaller matrices, the differences between configurations are less pronounced, and using more processes can even increase the total time.

These measurements include several operations outside the multiplication itself, which explains why their values are much higher than the per-multiplication timings.

The results highlight the importance of distinguishing the computational kernel from the complete application execution.

5 Discussion

The experimental results show that MPI can reduce matrix multiplication time by distributing the computation among several processes. The improvement becomes clearer for larger matrices, where each process performs more work and the relative impact of coordination costs is smaller.

However, the observed speedup remains below the ideal value. For $n = 1000$, eight processes achieve a speedup of 3.73 rather than 8. Interestingly, the computation phase alone decreases from 340.77 ms to 87.98 ms, corresponding to a speedup of approximately 3.87. This suggests that the limited scalability cannot be explained by communication overhead alone, since the computation itself does not scale linearly.

One possible explanation is the heterogeneous architecture of the Intel Core Ultra 7 155H, which includes six performance cores, eight efficient cores and two low-power efficient cores. Processes running on different types of cores may have different execution times. Shared memory bandwidth and changes in CPU frequency could also affect performance. However, process affinity and CPU frequency were not measured, so these remain possible explanations rather than confirmed causes.

The communication measurements also require careful interpretation. Gather shows an increasing duration as more processes are used, but this operation may include time spent waiting for slower processes. Therefore, its measured duration does not necessarily represent the actual cost of transferring the results.

The amount of data exchanged also increases with matrix size and process count, although the recorded phase durations do not isolate the cost of transferring these bytes.

Memory usage presents another limitation. Although each process receives fewer rows of matrix A, it still stores a complete copy of B. As a result, the sum of individual peak working sets increases with the number of processes. The root process also stores the complete input and output matrices and loads the expected result during validation, so its memory requirements differ from those of the other processes.

The comparison with OpenMP illustrates the differences between shared-memory and message-passing approaches. Both improve performance compared with the sequential implementation for larger matrices. For $n = 1000$, OpenMP with eight threads achieves a lower execution time than MPI with eight processes. With four workers, the measured times are similar, although this pattern does not hold for every matrix size.

These comparisons must be interpreted carefully. The MPI timings include data distribution, result collection and synchronisation, while the sequential and OpenMP measurements from Assignment 3 use different timing boundaries. Consequently, the observed differences cannot be attributed exclusively to the programming frameworks.

Finally, all MPI processes were executed on a single computer, sharing the same physical memory and hardware resources. The study therefore evaluates local multi-process performance rather than communication across independent machines. The largest tested matrix size was $n = 1000$, and the results may differ for larger workloads or other hardware configurations.

6 Conclusions

The experiment investigated the scalability of dense matrix multiplication using MPI with different process counts.

The results show that increasing the number of processes generally reduces execution time for larger matrices. For $n = 1000$, the median time decreased from 341.60 ms with one process to 91.52 ms with eight, corresponding to a speedup of 3.73.

However, the improvement was not proportional to the number of processes. Smaller workloads showed limited benefits, and the memory measurements revealed additional costs associated with multiple processes.

The comparison with OpenMP showed that both approaches can improve performance, although OpenMP with eight threads achieved the lowest measured time for the largest tested matrix.

Overall, MPI provided useful acceleration under the tested conditions, but communication, memory requirements and execution overhead limited its scalability.

Since the experiments were performed on a single computer, the conclusions apply to local multi-process execution. Evaluating MPI across multiple physical machines would be a possible extension of this work.

References

[1] Message Passing Interface Forum, *MPI: A Message-Passing Interface Standard, Version 4.1*, 2023. Available: `https://www.mpi-forum.org/docs/mpi-4.1/`

[2] Microsoft, *GetProcessMemoryInfo function (psapi.h)*, Microsoft Learn. Available: `https://learn.microsoft.com/en-us/windows/win32/api/psapi/nf-psapi-getprocessmemoryinfo`

Reproducibility Materials

The source code, input references, benchmark scripts, raw CSV measurements and generated figures are provided in the Assignment 4 directory of the project repository.

The main implementation is `c/distributed.c`. The scripts used for local benchmarking, end-to-end measurements and figure generation are stored in `scripts/`.

The `README.md` contains the compilation and execution instructions.

AI Usage Disclosure

ChatGPT was used to clarify technical concepts, assist with debugging and review parts of the report and README documentation. Claude was also used to review the final report and suggest improvements to the analysis and reproducibility. All experimental results were obtained and validated through the implemented code.

A Complete MPI Timing Results

Table 3 presents the median execution time and interquartile range (IQR) for all 20 configurations. Each configuration was measured five times, excluding one warm-up run.

Table 3: MPI execution time and variability across all configurations.

Matrix size	Processes	Median (ms)	IQR (ms)
100	1	0.3524	0.0212
100	2	0.3157	0.0229
100	4	0.2271	0.0112
100	8	0.3843	0.3404
300	1	10.3178	1.3141
300	2	7.6556	0.2782
300	4	4.7100	0.5168
300	8	3.5028	0.7431
500	1	49.3602	0.7965
500	2	39.6393	6.0807
500	4	21.2558	10.1067
500	8	13.7095	2.3132
750	1	149.5514	8.7713
750	2	81.6463	1.0016
750	4	55.0210	4.0102
750	8	39.4581	0.6924
1000	1	341.5997	12.2407
1000	2	185.8383	4.5838
1000	4	111.5660	5.4484
1000	8	91.5206	10.8812

Most configurations show relatively small variability, although some results are less stable. For $n = 100$ with eight processes, the IQR reaches 0.3404 ms, close to the median of 0.3843 ms. A similar effect appears for $n = 500$ with four processes, where the IQR is 10.1067 ms compared with a median of 21.2558 ms.

These variations suggest that execution conditions can affect the measurements, particularly for short workloads or certain process configurations. The available data does not allow their exact causes to be identified.